PDDL Planning Hands-On: robot de taller que cambia una rueda

Inteligencia Artificial – MUAR, ETSII-UPM · Gonzalo Parot Leon · Octubre 2026

1. Sistema robótico y escenario

Se modela un robot móvil de dos brazos, robi, que asiste a los mecánicos de un taller (proyecto METATOOL). El taller tiene varios lugares (almacén, mesa de herramientas, panel, zona del coche e impresora 3D) y el robot debe completar tareas relacionadas con el **cambio de una rueda pinchada**: cambiar la rueda, quitar una tuerca de medida poco habitual o quitar una tuerca antirrobo. Cada tarea exige una herramienta adecuada, y el escenario se plantea en cuatro problemas, uno por actividad: **usar**, **buscar**, **fabricar** e **inventar** herramientas.

2. Decisiones de modelado

- **Jerarquía de tipos.** Todo lo que ocupa un sitio hereda de localizable, de modo que un único predicado (en ?x ?l) describe la posición de robots, herramientas, material y tareas. La pertenencia (un brazo es de un robot) se modela con un predicado y no con un subtipo, porque el subtipo expresa la relación «es un», no «es parte de».

```
object
 +-- lugar (almacen, mesa, zona-coche, impresora...)  +--
localizable (todo lo que esta en algun lugar)
 | +-- robot +-- herramienta +-- material +-- tarea  +--
brazo +-- diseno +-- principio +-- capacidad
```

Figura 1. Jerarquía de tipos del dominio.

- **Capacidades en lugar de herramientas concretas.** Una tarea no pide «la llave de cruz», sino una capacidad (girar-19), y cualquier herramienta que la proporcione sirve. Esta abstracción es la que permite que una herramienta fabricada o inventada sustituya a una que no existe.
- **Mundo frente a conocimiento del robot.** (en ?h ?l) indica dónde está realmente una herramienta y (localizada ?h) si el robot lo sabe. La acción buscar convierte lo primero en lo segundo, y es necesaria para poder coger la herramienta.
- **Fabricar e inventar.** Una herramienta que aún no existe se declara con (por-fabricar ?h). fabricar la crea en una estación de fabricación a partir de un diseño, consumiendo material. inventar convierte un diseño vacío en un diseño para una capacidad, si el robot conoce dos principios que se combinan para obtenerla.
- **Dos brazos.** En usar, un brazo aplica la herramienta y el otro debe estar libre para sujetar la pieza. Como un brazo no puede estar a la vez libre y sujetando algo, se garantiza que son dos brazos distintos.
- **Compatibilidad.** Solo se usan :strips y :typing, para que el dominio funcione con cualquier planificador clásico.

3. Dominio

El dominio tiene 14 predicados y 7 acciones: tres de movimiento y manipulación (mover, coger, soltar) y una por cada actividad del enunciado.

```
; Dominio: robot de dos brazos en un taller mecanico (METATOOL)
(define (domain taller)
 (:requirements :strips :typing)
(:types
 lugar localizable brazo diseno principio capacidad - object
 robot herramienta material tarea - localizable)

 (:predicates
 (en ?x - localizable ?l - lugar) ; algo esta en un lugar
 (brazo-de ?b - brazo ?r - robot) ; el brazo pertenece al robot  (libre ?b -
brazo) ; el brazo no sujeta nada
 (sujeta ?b - brazo ?h - herramienta) ; el brazo sujeta una herramienta
(localizada ?h - herramienta) ; el robot sabe donde esta  (proporciona ?h -
herramienta ?c - capacidad) ; para que sirve la herramienta  (requiere ?t - tarea
?c - capacidad) ; que necesita la tarea
 (hecha ?t - tarea) ; la tarea esta terminada
 (estacion-fabricacion ?l - lugar) ; aqui hay una impresora 3D  (por-fabricar ?h
- herramienta) ; herramienta que aun no existe  (diseno-para ?d - diseno ?c -
capacidad) ; el diseno da esa capacidad  (diseno-vacio ?d - diseno) ; idea aun no
inventada
 (conoce ?r - robot ?p - principio) ; principio que el robot conoce  (combinan
?p1 - principio ?p2 - principio ?c - capacidad))

 (:action mover
 :parameters (?r - robot ?desde - lugar ?hasta - lugar)
 :precondition (en ?r ?desde)
 :effect (and (not (en ?r ?desde)) (en ?r ?hasta)))

 ; FIND: el robot inspecciona un lugar y descubre la herramienta
 (:action buscar
 :parameters (?r - robot ?h - herramienta ?l - lugar)
 :precondition (and (en ?r ?l) (en ?h ?l))
 :effect (localizada ?h))

 (:action coger
 :parameters (?r - robot ?b - brazo ?h - herramienta ?l - lugar)
 :precondition (and (en ?r ?l) (en ?h ?l) (localizada ?h)
 (brazo-de ?b ?r) (libre ?b))
 :effect (and (sujeta ?b ?h) (not (libre ?b)) (not (en ?h ?l))))

 (:action soltar
 :parameters (?r - robot ?b - brazo ?h - herramienta ?l - lugar)
 :precondition (and (en ?r ?l) (brazo-de ?b ?r) (sujeta ?b ?h))
 :effect (and (en ?h ?l) (libre ?b) (not (sujeta ?b ?h))))

 ; USE: un brazo aplica la herramienta y el otro sujeta la pieza
 (:action usar
 :parameters (?r - robot ?b1 - brazo ?b2 - brazo ?h - herramienta
 ?t - tarea ?c - capacidad ?l - lugar)
 :precondition (and (en ?r ?l) (en ?t ?l)
 (brazo-de ?b1 ?r) (brazo-de ?b2 ?r)
 (sujeta ?b1 ?h) (libre ?b2)
 (proporciona ?h ?c) (requiere ?t ?c))
 :effect (hecha ?t))

 ; MAKE: imprimir una herramienta a partir de un diseno
 (:action fabricar
 :parameters (?r - robot ?d - diseno ?h - herramienta ?m - material
 ?c - capacidad ?l - lugar)
 :precondition (and (en ?r ?l) (estacion-fabricacion ?l) (en ?m ?l)
 (diseno-para ?d ?c) (por-fabricar ?h))
 :effect (and (en ?h ?l) (localizada ?h) (proporciona ?h ?c)
 (not (por-fabricar ?h)) (not (en ?m ?l))))

 ; INVENT: combinar dos principios conocidos en un diseno nuevo
 (:action inventar
 :parameters (?r - robot ?d - diseno ?p1 - principio ?p2 - principio
 ?c - capacidad)
 :precondition (and (conoce ?r ?p1) (conoce ?r ?p2)
 (combinan ?p1 ?p2 ?c) (diseno-vacio ?d))
 :effect (and (diseno-para ?d ?c) (not (diseno-vacio ?d))))
)
```

Listado 1. domain.pddl

4. Problemas

En los cuatro problemas el robot empieza en el almacén con los dos brazos libres, y el objetivo es completar una tarea en la zona del coche. Cada problema elimina algo para forzar una actividad distinta: como el planificador solo hace lo imprescindible, nunca fabricaría ni inventaría una herramienta si ya tuviera una que sirve.

Problema	Tarea y capacidad	Situación de partida
P1 · usar	cambiar-rueda (girar-19)	La llave de cruz está en la mesa y el robot lo sabe.
P2 · buscar	cambiar-rueda (girar-19)	La llave está en la mesa, pero no está localizada. Hay un destornillador como distractor.
P3 · fabricar	quitar-tuerca-22 (girar-22)	Ninguna herramienta sirve, pero existe un diseño de adaptador y hay filamento en la impresora.
P4 · inventar	quitar-antirrobo (girar-antirrobo)	No hay herramienta ni diseño; el robot conoce dos principios que se combinan.

Tabla 1. Resumen de los cuatro problemas.

```
; P1 (USE): el robot sabe donde esta la llave de cruz
(define (problem p1-usar)
 (:domain taller)
 (:objects
 robi - robot
 izq der - brazo
 almacen mesa zona-coche - lugar
 llave-cruz - herramienta
 cambiar-rueda - tarea
 girar-19 - capacidad)
 (:init
 (en robi almacen)
 (brazo-de izq robi) (brazo-de der robi)
 (libre izq) (libre der)
 (en llave-cruz mesa) (localizada llave-cruz)
 (proporciona llave-cruz girar-19)
 (en cambiar-rueda zona-coche)
 (requiere cambiar-rueda girar-19))
 (:goal (hecha cambiar-rueda)))
```

Listado. p1-usar.pddl

```
; P2 (FIND): el robot no sabe donde esta la llave de cruz
(define (problem p2-buscar)
 (:domain taller)
 (:objects
 robi - robot
 izq der - brazo
 almacen mesa panel zona-coche - lugar
 llave-cruz destornillador - herramienta
 cambiar-rueda - tarea
 girar-19 atornillar - capacidad)
 (:init
 (en robi almacen)
 (brazo-de izq robi) (brazo-de der robi)
 (libre izq) (libre der)
 (en llave-cruz mesa) ; no esta localizada
 (proporciona llave-cruz girar-19)
 (en destornillador panel) ; tampoco, y no sirve
 (proporciona destornillador atornillar)
 (en cambiar-rueda zona-coche)
 (requiere cambiar-rueda girar-19))
 (:goal (hecha cambiar-rueda)))
```

Listado. p2-buscar.pddl

```
; P3 (MAKE): tuerca de 22 mm sin llave, pero hay un diseno de adaptador
(define (problem p3-fabricar)
 (:domain taller)
 (:objects
 robi - robot
 izq der - brazo
 almacen zona-coche impresora - lugar
 adaptador-22 - herramienta
 filamento - material
 d-adaptador - diseno
 quitar-tuerca-22 - tarea
 girar-22 - capacidad)
 (:init
 (en robi almacen)
 (brazo-de izq robi) (brazo-de der robi)
 (libre izq) (libre der)
 (estacion-fabricacion impresora) (en filamento impresora)
 (diseno-para d-adaptador girar-22)
 (por-fabricar adaptador-22)
 (en quitar-tuerca-22 zona-coche)
 (requiere quitar-tuerca-22 girar-22))
 (:goal (hecha quitar-tuerca-22)))
```

Listado. p3-fabricar.pddl

```
; P4 (INVENT): tuerca antirrobo sin herramienta ni diseno
(define (problem p4-inventar)
 (:domain taller)
 (:objects
 robi - robot
 izq der - brazo
 almacen zona-coche impresora - lugar
 extractor - herramienta
 filamento - material
 d-nuevo - diseno
 agarre-presion giro-palanca - principio
 quitar-antirrobo - tarea
 girar-antirrobo - capacidad)
 (:init
 (en robi almacen)
 (brazo-de izq robi) (brazo-de der robi)
 (libre izq) (libre der)
 (estacion-fabricacion impresora) (en filamento impresora)
 (por-fabricar extractor)
 (diseno-vacio d-nuevo)
 (conoce robi agarre-presion) (conoce robi giro-palanca)
 (combinan agarre-presion giro-palanca girar-antirrobo)
 (en quitar-antirrobo zona-coche)
 (requiere quitar-antirrobo girar-antirrobo))
 (:goal (hecha quitar-antirrobo)))
```

Listado. p4-inventar.pddl

5. Planes obtenidos e interpretación

Los problemas se resolvieron con el planificador **pyperplan** mediante búsqueda en anchura, que garantiza planes de longitud mínima. Los cuatro tienen solución, con 4, 5, 5 y 6 acciones.

```
; P1 usar
0 (mover robi almacen mesa)
1 (coger robi izq llave-cruz mesa)
2 (mover robi mesa zona-coche)
3 (usar robi izq der llave-cruz cambiar-rueda girar-19 zona-coche)

; P2 buscar
0 (mover robi almacen mesa)
1 (buscar robi llave-cruz mesa)
2 (coger robi der llave-cruz mesa)
3 (mover robi mesa zona-coche)
4 (usar robi der izq llave-cruz cambiar-rueda girar-19 zona-coche)

; P3 fabricar
0 (mover robi almacen impresora)
1 (fabricar robi d-adaptador adaptador-22 filamento girar-22 impresora)
2 (coger robi izq adaptador-22 impresora)
3 (mover robi impresora zona-coche)
4 (usar robi izq der adaptador-22 quitar-tuerca-22 girar-22 zona-coche)

; P4 inventar
0 (mover robi almacen impresora)
1 (inventar robi d-nuevo agarre-presion giro-palanca girar-antirrobo)
2 (fabricar robi d-nuevo extractor filamento girar-antirrobo impresora)
3 (coger robi der extractor impresora)
4 (mover robi impresora zona-coche)
5 (usar robi der izq extractor quitar-antirrobo girar-antirrobo zona-coche)
```

Listado. Planes obtenidos.

- **P1 (usar).** El robot va a la mesa, coge la llave con un brazo, va al coche y la usa con el otro brazo libre. Es el plan mínimo: solo dos desplazamientos.
- **P2 (buscar).** Idéntico a P1, pero con buscar antes de coger. El destornillador no se busca porque no proporciona la capacidad necesaria: el planificador solo adquiere el conocimiento que le hace falta.
- **P3 (fabricar).** Como ninguna herramienta proporciona girar-22, el robot va a la impresora, fabrica el adaptador a partir del diseño y lo lleva al coche.
- **P4 (inventar).** Sin herramienta ni diseño, el planificador encadena por sí solo inventar → fabricar → usar: combina agarre por presión y giro por palanca en un diseño nuevo, lo imprime y lo usa.

6. Limitaciones y posibles extensiones

El modelo es determinista y de mundo cerrado: el planificador conoce de antemano dónde está la herramienta «buscada», así que buscar representa la adquisición de conocimiento, pero no una búsqueda real con incertidumbre, que requeriría planificación contingente. Además, cada tarea se resuelve con una sola acción usar (el cambio de rueda no se descompone en sus pasos), las acciones no tienen duración ni coste y el material se consume entero. Extensiones naturales serían descomponer las tareas en subacciones y usar acciones durativas y funciones numéricas (batería del robot, tiempo de impresión), como describe la sección 6 de la guía.